🛠 Bash for Data Engineers

sed • awk • cut • Functions

Advanced Session | Data Engineering Track

1. cut — Extract Columns from Text

cut is the simplest of the three tools. It slices columns or characters out of each line of text. Perfect for quickly extracting a specific field from a CSV or delimited file.

1.1 Basic Syntax

```
cut [OPTIONS] filename

  -d   Delimiter (default is TAB)
  -f   Field number(s) to extract
  -c   Character position(s) to extract
```

1.2 Working with CSV / Delimited Files

Imagine you have a file called employees.csv:

```
id,name,department,salary
1,Ahmed,Engineering,85000
2,Sara,Data,92000
3,Omar,DevOps,78000
4,Lina,Data,95000
```

Extract one column — just the names:

```
cut -d"," -f2 employees.csv
```

Output:

```
name
Ahmed
Sara
Omar
Lina
```

Extract multiple columns — name and salary:

```
cut -d"," -f2,4 employees.csv
```

Output:

```
name,salary
Ahmed,85000
Sara,92000
Omar,78000
Lina,95000
```

Extract a range of columns — fields 2 to 4:

```
cut -d"," -f2-4 employees.csv
```

Output:

```
name,department,salary
Ahmed,Engineering,85000
Sara,Data,92000
```

Extract by character position (first 5 chars of each line):

```
cut -c1-5 employees.csv
```

Output:

```
id,na
1,Ahm
2,Sara
3,Omar
4,Lina
```

Skip the header line and extract department:

```
tail -n +2 employees.csv | cut -d"," -f3
```

Output:

```
Engineering
Data
DevOps
Data
```

1.3 cut Quick Reference

Option	Meaning	Example
`-d','`	Set comma as delimiter	cut -d',' -f1
`-f2`	Extract field 2	cut -d',' -f2 file.csv
`-f2,4`	Extract fields 2 and 4	cut -d',' -f2,4 file.csv
`-f2-5`	Extract fields 2 to 5	cut -d',' -f2-5 file.csv
`-c1-8`	Extract chars 1 to 8	cut -c1-8 file.txt

cut is fast and simple but limited — it cannot reorder columns or do math. For that, use awk.

2. sed — Stream Editor

sed processes text line by line and is mainly used for find & replace, deletion, and insertion. It is the go-to tool for cleaning and transforming text data in pipelines.

2.1 Basic Syntax

```
sed [OPTIONS] 'command' filename

  -i    Edit the file in-place (modifies the original file)
  -n    Suppress automatic printing
  -e    Run multiple commands
```

2.2 Find & Replace

The most common sed command is substitution: s/old/new/

```
# Replace first occurrence per line
sed 's/Data/Analytics/' employees.csv

# Replace ALL occurrences per line (add g flag)
sed 's/Data/Analytics/g' employees.csv

# Case-insensitive replace (add I flag)
sed 's/data/Analytics/gI' employees.csv
```

Output:

```
id,name,department,salary
1,Ahmed,Engineering,85000
2,Sara,Analytics,92000
3,Omar,DevOps,78000
4,Lina,Analytics,95000
```

Replace and save to the same file (in-place):

```
sed -i 's/Data/Analytics/g' employees.csv
```

⚠ **The -i flag modifies your original file permanently. Always keep a backup!**

Replace only in lines that match a pattern:

```
# Only replace salary in lines containing 'Data'
sed '/Data/s/9[0-9]*/CONFIDENTIAL/' employees.csv
```

2.3 Delete Lines

```
# Delete line number 1 (the header)
sed '1d' employees.csv

# Delete lines matching a pattern
sed '/DevOps/d' employees.csv

# Delete blank/empty lines
sed '/^$/d' file.txt

# Delete lines 2 to 4
sed '2,4d' employees.csv
```

2.4 Print Specific Lines

```
# Print only line 3
sed -n '3p' employees.csv

# Print lines 2 to 4
sed -n '2,4p' employees.csv

# Print lines matching a pattern
sed -n '/Data/p' employees.csv
```

Output:

```
2,Sara,Data,92000
4,Lina,Data,95000
```

2.5 Insert & Append Text

```
# Insert a line BEFORE line 1 (add a title)
sed '1i\# Employee Report' employees.csv

# Append a line AFTER the last line
sed '$a\# End of file' employees.csv

# Add header to a file that is missing one
sed '1i\id,name,department,salary' data_no_header.csv
```

2.6 Multiple Commands with -e

```
# Replace 'Data' with 'Analytics' AND delete DevOps lines
sed -e 's/Data/Analytics/g' -e '/DevOps/d' employees.csv
```

2.7 Real Data Engineering Examples

Clean a log file — remove timestamps and keep only the message:

```
# log.txt line: [2024-01-15 08:30:22] ERROR: Connection failed
sed 's/\[.*\] //' log.txt
```

Output:

```
ERROR: Connection failed
```

Convert CSV delimiter from comma to pipe:

```
sed 's/,/|/g' employees.csv
```

Output:

```
id|name|department|salary
1|Ahmed|Engineering|85000
```

Remove all lines starting with # (comments in config files):

```
sed '/^#/d' config.conf
```

2.8 sed Quick Reference

Command	Meaning	Example
`s/old/new/`	Replace first match per line	sed 's/foo/bar/' f.txt
`s/old/new/g`	Replace all matches per line	sed 's/foo/bar/g' f.txt
`Nd`	Delete line N	sed '1d' f.txt
`/pat/d`	Delete lines matching pattern	sed '/error/d' f.txt
`-n 'Np'`	Print only line N	sed -n '5p' f.txt
`/pat/p`	Print lines matching pattern	sed -n '/Data/p' f.txt
`Ni\text`	Insert text before line N	sed '1i\header' f.txt
`s/old/new/I`	Case-insensitive replace	sed 's/data/X/gI' f.txt
`-e cmd1 -e cmd2`	Run multiple commands	sed -e 's/a/b/' -e '1d'

3. awk — The Data Processing Powerhouse

awk is a full mini-programming language built for structured text. It processes files field-by-field and line-by-line. For data engineers it is essential — it can filter, transform, aggregate, and compute on the fly.

3.1 How awk Thinks

awk reads a file line by line. Each line is called a record. Each word/column in a line is called a field.

```
# For this line:   1,Ahmed,Data,92000
#
#   $0  = the whole line
#   $1  = 1
#   $2  = Ahmed
#   $3  = Data
#   $4  = 92000
#   NR  = current line number
#   NF  = number of fields in this line
```

3.2 Basic Syntax

```
awk 'BEGIN { setup } /pattern/ { action } END { teardown }' filename

  BEGIN  runs once before processing any lines
  END    runs once after processing all lines
  { }    runs for every line (or matching lines)
```

3.3 Printing Fields

```
# Print field 2 (name) from each line
awk -F"," '{print $2}' employees.csv

# Print name and salary with a label
awk -F"," '{print $2, "earns", $4}' employees.csv

# Print with custom separator
awk -F"," 'BEGIN{OFS=" | "} {print $2,$3,$4}' employees.csv

# Skip the header (NR > 1)
awk -F"," 'NR>1 {print $2,$4}' employees.csv
```

Output:

```
Ahmed earns 85000
Sara earns 92000
Omar earns 78000
Lina earns 95000
```

3.4 Filtering Rows

```
# Print lines where department is Data
awk -F"," '$3 == "Data"' employees.csv

# Print lines where salary > 90000
awk -F"," '$4 > 90000' employees.csv

# Print lines where name contains "a" (regex)
awk -F"," '$2 ~ /a/' employees.csv

# Combine conditions (AND)
awk -F"," '$3 == "Data" && $4 > 90000' employees.csv
```

Output:

```
2,Sara,Data,92000
4,Lina,Data,95000
```

3.5 Calculations & Aggregation

Sum of all salaries:

```
awk -F"," 'NR>1 {sum += $4} END {print "Total salary:", sum}'
employees.csv
```

Output:

```
Total salary: 350000
```

Average salary:

```
awk -F"," 'NR>1 {sum += $4; count++} END {print "Average:", sum/count}'
employees.csv
```

Output:

```
Average: 87500
```

Count employees per department:

```
awk -F"," 'NR>1 {dept[$3]++} END {for (d in dept) print d, dept[d]}'
employees.csv
```

Output:

```
Engineering 1
Data 2
DevOps 1
```

Max salary:

```
awk -F"," 'NR>1 {if($4>max) max=$4} END {print "Max salary:", max}'
employees.csv
```

Output:

```
Max salary: 95000
```

3.6 BEGIN and END Blocks

Add a header and footer to output:

```
awk -F"," '
BEGIN {
    print "===== Employee Report ======"
    print "Name\t\tDepartment\tSalary"
    print "----\t\t----------\t------"
}
NR>1 {
    printf "%-12s\t%-12s\t%s\n", $2, $3, $4
}
END {
    print "==========================="
}' employees.csv
```

Output:

```
===== Employee Report =====
Name          Department      Salary
----          ----------      ------
Ahmed         Engineering     85000
Sara          Data            92000
Omar          DevOps          78000
Lina          Data            95000
===========================
```

3.7 awk for Log File Analysis

Given a log file: logs.txt

```
[2024-01-15 08:30:22] ERROR: Connection timeout
[2024-01-15 08:31:05] INFO: Job started
[2024-01-15 08:35:10] ERROR: Disk full
[2024-01-15 08:36:00] INFO: Job completed
[2024-01-15 08:36:01] WARNING: Low memory
```

Count each log level:

```
awk '{count[$2]++} END {for (level in count) print level, count[level]}'
logs.txt
```

Output:

```
ERROR: 2
INFO: 2
WARNING: 1
```

Print only ERROR lines:

```
awk '/ERROR/' logs.txt
```

Extract just the message after the level:

```
awk '{$1=$2=""; print $0}' logs.txt
```

3.8 awk Quick Reference

Variable / Pattern	Meaning
`$0`	The entire current line
`$1, $2, $NF`	Field 1, field 2, last field
`NR`	Current line (record) number
`NF`	Number of fields in current line
`FS / -F`	Input field separator (default: space)
`OFS`	Output field separator
`BEGIN { }`	Runs once before any lines are read
`END { }`	Runs once after all lines are read
`/pattern/`	Match lines containing pattern
`$2 == "val"`	Match lines where field 2 equals value
`$3 ~ /regex/`	Match lines where field 3 matches regex
`$4 > 1000`	Match lines where field 4 > 1000

4. Combining cut, sed & awk in Pipelines

The real power comes from chaining these tools together using pipes (|). Each tool does one job and passes the result to the next.

Pipeline 1 — Clean CSV then extract a column:

```
# Remove blank lines, then get just the names
sed '/^$/d' employees.csv | cut -d',' -f2
```

Pipeline 2 — Replace header then compute average salary:

```
sed 's/salary/SALARY/' employees.csv | awk -F"," 'NR>1{s+=$4}END{print
"Avg:", s/(NR-1)}'
```

Pipeline 3 — Full data cleaning pipeline:

```
cat raw_data.csv          \
  | sed '/^#/d'           \  # Remove comment lines
  | sed '/^$/d'           \  # Remove blank lines
  | sed 's/ //g'          \  # Remove all spaces
  | awk -F"," '$4>0'    \  # Keep rows with positive salary
  | cut -d"," -f2,3,4        # Extract name, dept, salary
```

5. Bash Functions — Local vs Global Variables

Functions let you reuse code. Understanding variable scope (local vs global) is critical — a bug caused by a global variable accidentally being modified is hard to trace.

5.1 Defining and Calling Functions

```
# Style 1
greet() {
    echo "Hello, $1!"
```

```
}

# Style 2 (same thing)
function greet() {
    echo "Hello, $1!"
}

# Call the function
greet "Ahmed"    # => Hello, Ahmed!
```

5.2 Global Variables (Default)

By default, ALL variables in Bash are global. This means a variable set inside a function is visible everywhere — even after the function returns.

```
#!/bin/bash

RESULT="nothing"   # global variable

calculate() {
    RESULT=42       # modifies the GLOBAL variable
}

echo "Before: $RESULT"  # => nothing
calculate
echo "After:  $RESULT"  # => 42   (changed globally!)
```

> ⚠ **If two functions use the same variable name without 'local', they will overwrite each other. This causes hard-to-find bugs.**

5.3 Local Variables (Best Practice)

Use the 'local' keyword to make a variable private to the function. It disappears when the function ends.

```
#!/bin/bash

RESULT="global value"

calculate() {
    local RESULT=42   # local — only exists inside this function
    echo "Inside function: $RESULT"   # => 42
}

echo "Before: $RESULT"   # => global value
```

```
calculate
echo "After:  $RESULT"   # => global value  (unchanged!)
```

Output:

```
Before: global value
Inside function: 42
After:  global value
```

5.4 Side-by-Side Comparison

Without local (dangerous)

```
#!/bin/bash

NAME="Global Ahmed"

process() {
    NAME="Modified!"
    echo $NAME
}

echo $NAME  # Global Ahmed
process
# => Modified!
echo $NAME  # => Modified!
#   ^ BUG: global was changed!
```

With local (safe)

```
#!/bin/bash

NAME="Global Ahmed"

process() {
    local NAME="Modified!"
    echo $NAME
}

echo $NAME  # Global Ahmed
process
# => Modified!
echo $NAME  # => Global Ahmed
#   ^ SAFE: global unchanged
```

5.5 Returning Values from Functions

Bash functions cannot return strings with 'return' (it only returns exit codes 0–255). The correct way is to echo the value and capture it.

```
#!/bin/bash

# Method 1: echo + command substitution (recommended)
get_department() {
    local emp_id=$1
    if [[ $emp_id -eq 1 ]]; then
        echo "Engineering"
    else
        echo "Unknown"
    fi
}

DEPT=$(get_department 1)
echo "Department: $DEPT"   # => Department: Engineering
```

```
# Method 2: use a global variable as 'return container'
calculate_bonus() {
    local salary=$1
    local rate=$2
    BONUS=$(( salary * rate / 100 ))  # writes to global BONUS
}

calculate_bonus 92000 10
echo "Bonus: $BONUS"   # => Bonus: 9200
```

5.6 A Complete Practical Example

A data processing script using functions properly with local variables:

```
#!/bin/bash

# ── Global config ──────────────────────────
INPUT_FILE="employees.csv"
REPORT_FILE="report.txt"

# ── Functions ──────────────────────────────
get_avg_salary() {
    local file=$1
    local avg=$(awk -F"," 'NR>1{s+=$4;c++}END{printf "%.0f",s/c}'
"$file")
    echo $avg
}

count_by_dept() {
    local file=$1
    local dept=$2
    local count=$(awk -F"," -v d="$dept" 'NR>1 && $3==d{c++}END{print
c+0}' "$file")
    echo $count
}

generate_report() {
    local file=$1
    local outfile=$2
    local avg=$(get_avg_salary $file)
    local data_count=$(count_by_dept $file Data)
    local eng_count=$(count_by_dept $file Engineering)
    {
        echo "===== Salary Report ======"
        echo "Average Salary : $avg"
        echo "Data Team      : $data_count employees"
        echo "Engineering    : $eng_count employees"
```

```
        echo "========================"
    } > $outfile
    echo "Report saved to $outfile"
}

# ── Main ──────────────────────────────────
generate_report $INPUT_FILE $REPORT_FILE
```

Output:

```
===== Salary Report =====
Average Salary : 87500
Data Team      : 2 employees
Engineering    : 1 employees
========================
Report saved to report.txt
```

5.7 Local vs Global — Rules to Remember

Rule	Explanation
`Always use local inside functions`	Prevents accidental overwriting of variables elsewhere
`Use global for config/constants`	Things like INPUT_FILE, LOG_DIR set once at the top
`Return strings via echo + $()`	Use RESULT=$(my_func) to capture output
`Return status via return 0 / return 1`	0 = success, non-zero = error — use in if statements
`Name local vars clearly`	local input_file=$1 is clearer than local f=$1

6. Practice Exercises

Exercise 1 — cut

1. From employees.csv, extract only the name and department columns
2. Skip the header line using tail, then extract the salary column
3. Extract characters 1–3 from each line

Exercise 2 — sed

4. Replace all occurrences of 'Data' with 'Analytics' in employees.csv
5. Delete the header line from the file output
6. Print only lines that contain salaries above 90000 (hint: use /9[0-9][0-9][0-9][0-9]/p)

Exercise 3 — awk

7. Print name and salary for employees earning more than 88000
8. Count how many employees are in each department
9. Calculate the total salary bill for the Data department only

Exercise 4 — Functions with local variables

10. Write a function is_high_earner that takes a salary and prints 'High' if > 90000, else 'Normal'
11. Write a function get_full_info that takes an employee ID, reads employees.csv, and returns their name and department
12. Make sure ALL internal variables use 'local' — test that global variables are not modified

7. Summary Cheat Sheet

Tool	Best For	Typical Usage
`cut`	Extract columns by position	cut -d',' -f2,4 file.csv
`sed`	Find & replace, delete, insert lines	sed 's/old/new/g' file.txt
`awk`	Filter, compute, aggregate fields	awk -F',' '$4>90000' file.csv
`local`	Scoped function variables	local var=value inside function
`echo+$()`	Return string from function	RESULT=$(my_func arg)
`pipe |`	Chain tools into a pipeline	sed ... | awk ... | cut ...

Master these tools and you can process almost any text data from the command line!